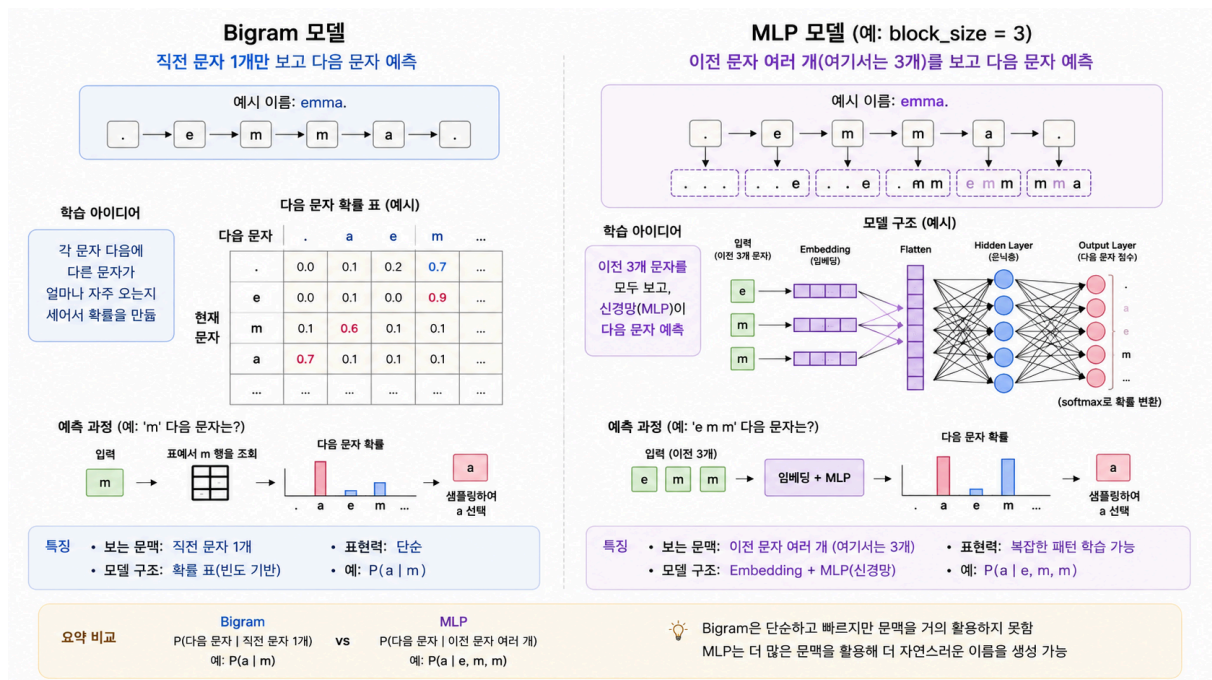


인공지능금융공학 - llm 만들기

Generating Mini GPT 2.0 project

Notebook 1

Bigram (block_size=1, 이전 문자 1개 보고 1개 예측)



Language Model 을 만들기 위한 기초적 흐름

1. 텍스트 데이터 → 숫자 데이터
2. data set 구성 (X,Y)
3. Model 학습
4. Token prediction
5. Loss 계산
6. backpropagation으로 학습
7. 새 텍스트 생성

(1) 텍스트 데이터 불러오기 & 텍스트 → 숫자 변환

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from pathlib import Path

if not Path("names.txt").exists():
    !wget -q https://raw.githubusercontent.com/karpathy/makemore/master/names.txt

words = open("names.txt", "r").read().splitlines()
chars = sorted(list(set("".join(words))))
chars = ['.'] + chars

stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}
vocab_size = len(stoi)
encoded_words = [[stoi[ch] for ch in w] for w in words]

print("num words:", len(words))
print("vocab_size:", vocab_size)
print(words[:10])
```

(2) Data set 학습 (Data loader)

block_size = 1 → 모델이 입력값 1개 보고 예측값 1개 예측

- 학습 데이터 만들기 (X,Y)

입력 X 정답 Y

. → e

e → m

m → m

m → a

a → .

→ 데이터를 1개씩 입력하고 학습시키면 너무 비효율적이므로 Data Loader를 통해 입력 데이터와 결과 데이터를 묶어서 학습한다.

```
loader = DataLoader(dataset, batch_size=32, shuffle=True)
```

(x = [5] / y = 13) → (x=[5],[13],[4] ... (32,1) / y = [13,6,3, ...] (32,))

x.shape(1,) 를 32개 묶어서 x.shape(32,1) 을 만들어서 학습시킨다.

```
class NamesContextDataset(Dataset):
    def __init__(self, encoded_words, block_size):
        self.X, self.Y = [], []
        for word in encoded_words:
            context = [0] * block_size
            for ix in word + [0]:
                self.X.append(context.copy())
                self.Y.append(ix)
                context = context[1:] + [ix]
        self.X = torch.tensor(self.X, dtype=torch.long)
        self.Y = torch.tensor(self.Y, dtype=torch.long)

    def __len__(self):
        return len(self.Y)

    def __getitem__(self, idx):
        return self.X[idx], self.Y[idx]

block_size = 1
dataset = NamesContextDataset(encoded_words, block_size)
loader = DataLoader(dataset, batch_size=32, shuffle=True)

x, y = dataset[1]
xb, yb = next(iter(loader))
print("single x:", x, x.shape)
print("single y:", y)
print("batch xb.shape:", xb.shape)
print("batch yb.shape:", yb.shape)
```

(3) Biagram Language Model

```
class BigramLanguageModel(nn.Module):
    def __init__(self, vocab_size):
        super().__init__()
        self.vocab_size = vocab_size
        self.W = nn.Parameter(torch.randn(vocab_size, vocab_size))

    def forward(self, x):
        x = x.view(-1) # (B,)
        x_onehot = F.one_hot(x, num_classes=self.vocab_size).float() # (B, V)
        logits = x_onehot @ self.W
        # (B, V)
        return logits

model = BigramLanguageModel(vocab_size)
logits = model(xb)
print("logits.shape:", logits.shape)
print("initial loss:", F.cross_entropy(logits, yb).item())
```

W의 의미

- W = (문자 종류 개수, 문자 종류 개수) 의 행렬, 현재 문자가 i 일때 그다음 문자가 j 일 점수
- 문자종류(.,a,b,...,z) 개수가 27개일 때 $W.shape(27,27)$
- $W[5, 13] \Rightarrow$ 현재 문자가 5일때, 다음 문자가 13일 점수

One hot encoding

- 텍스트 $\rightarrow$ 숫자로 바꾼 후 그 숫자를 문자 종류 개수에 맞춰서 벡터로 표현해주는 것
- 즉, 텍스트 $\rightarrow$ 숫자 $\rightarrow$ 벡터 ex. 5 $\rightarrow [0,0,0,0,1,0,...,0]$

$x.shape = (B,)$ $\rightarrow$ B = batch size

$x_onehot.shape = (B, V)$ $\rightarrow$ V = Vocab size

logits

```
x_onehot.shape = (32, 27)
W.shape = (27, 27)
logits.shape = (32, 27)
```

→ logit은 입력값 (32,) 32개에 대해 각각 그 입력값 다음에 올 문자 27개의 점수들을 출력함.

ex. logits[0] 은 첫번째 입력인 (1,)에 대해

다음 문자가 .일 점수

다음 문자가 a일 점수

다음 문자가 b일 점수

...

다음 문자가 z일 점수

를 출력함.

Loss 계산

```
loss = F.cross_entropy(logits, yb)
```

`cross_entropy` 는 모델의 예측이 정답과 얼마나 다른지 계산함.

학습의 목표는 이 Loss를 작게 만드는 것 !!

Dimension 차원에서 정리

현재 설정:

```
block_size=1
batch_size=32
vocab_size=V
```

데이터 하나:

```
x.shape= (1, )
y.shape= ()
```

batch:

```
xb.shape= (32,1)
yb.shape= (32,)
```

one-hot:

```
x_onehot.shape= (32,V)
```

가중치:

```
W.shape= (V,V)
```

행렬곱:

```
logits=x_onehot@W
```

결과:

```
logits.shape= (32,V)
```

정답:

```
yb.shape= (32,)
```

loss 계산:

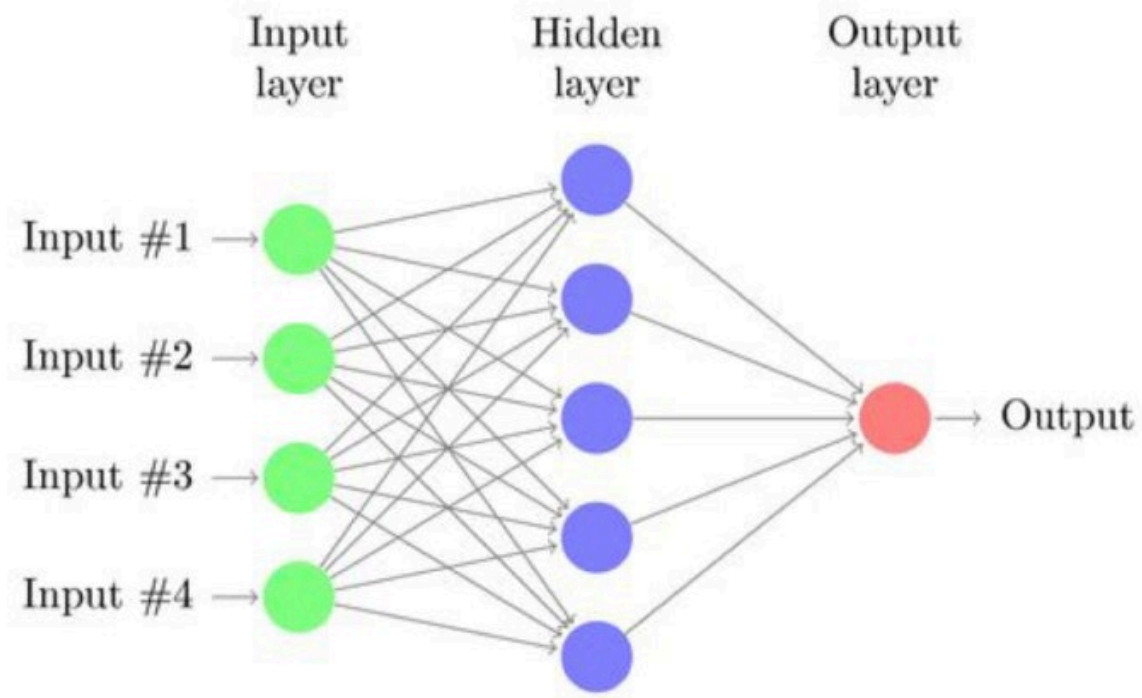
```
F.cross_entropy(logits,yb)
```

여기서 모델은 32개의 입력 각각에 대해 **v** 개 문자 중 정답 하나를 맞히는 문제를 푸는 거야.

(32, 1)	(32,)
2차원 배열 (32*1)	1차원 배열 (list 1개)
[[0], [1], ... , [31]]	[0,1, ..., 31]

Notebook 2

MLP (Multi layer Perceptron)



문자들(Input) → embedding → hidden layer → 다음 문자 확률(Output)

즉, 언어모델은 여러개의 문자들을 Input으로 받아서 다음에 올 문자의 확률(Output)을 예측하는 MLP 모델이다.

- Input 구조 emma → (...emma.)

입력 context	정답
...	e
..e	m
.em	m
emm	a

입력 context	정답
mma	.

ex)

`block_size = 1` → [e] : m

`block_size = 5` → [...em] : a

One-hot vs embedding

구분	One-hot	Embedding
기본 아이디어	문자를 긴 0/1 벡터로 표현	문자를 짧은 실수 벡터로 표현
예시	<code>e → [0,0,0,0,1,0,...]</code>	<code>e → [0.3, -1.2, 0.8]</code>
벡터 길이	문자 종류 수만큼 김	우리가 정한 embedding 차원만큼
예: 문자 27개	길이 27	예: 길이 2, 10, 64 등
값의 의미	해당 문자 위치만 1, 나머지는 0	모델이 학습한 숫자들
학습 여부	one-hot 자체는 학습되지 않음	embedding 값은 학습됨
효율성	비효율적. 대부분 0	효율적. 짧고 정보 압축 가능
문자 간 관계 표현	문자끼리 관계를 거의 표현 못 함	문자끼리 비슷한 패턴을 어느 정도 표현 가능
코드 예시	<code>F.one_hot(x, num_classes=vocab_size)</code>	<code>nn.Embedding(vocab_size, emb_dim)</code>
모델 입력 느낌	"이 문자는 몇 번 문자다"만 알려줌	"이 문자는 이런 특징 벡터다"라고 표현
GPT와의 관련성	기초 설명용으로 자주 사용	GPT에서 실제로 사용하는 방식

1. 기초 설정

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from pathlib import Path

if not Path("names.txt").exists():
```



```
!wget -q https://raw.githubusercontent.com/karpathy/makemore/master/names.txt
```

```
words = open("names.txt", "r").read().splitlines()
chars = sorted(list(set("".join(words))))
chars = ['.'] + chars
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}
vocab_size = len(stoi)
encoded_words = [[stoi[ch] for ch in w] for w in words]
```

- 라이브러리 불러오기

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from pathlib import Path
```

torch : pytorch 의 기본적인 라이브러리

torch.nn : MLP 층을 만들 때 쓰는 라이브러리

torch.nn.functional : MLP로 학습을 하는 과정에서 필요한 여러가지 함수들

Dataset, DataLoader : input 층으로 data를 입력 받았을 때, data들을 묶어서 학습시켜 주는 용도

2. Dataloader 규칙 설정

```
class NamesContextDataset(Dataset):
    def __init__(self, encoded_words, block_size):
        self.X, self.Y = [], [] # 입력 (X,Y) 설정
        for word in encoded_words:
            context = [0] * block_size # [0,0,0]
            for ix in word + [0]:
                self.X.append(context.copy())
                self.Y.append(ix)
```

```

        context = context[1:] + [ix]
    self.X = torch.tensor(self.X, dtype=torch.long)
    self.Y = torch.tensor(self.Y, dtype=torch.long)

    def __len__(self):
        return len(self.Y)

    def __getitem__(self, idx):
        return self.X[idx], self.Y[idx]

block_size = 3
dataset = NamesContextDataset(encoded_words, block_size)
loader = DataLoader(dataset, batch_size=64, shuffle=True)

xb, yb = next(iter(loader))
print("xb.shape:", xb.shape)
print("yb.shape:", yb.shape)

```

- 데이터 규칙

```

def __init__(self, encoded_words, block_size):
    self.X, self.Y = [], [] # 입력 (X,Y) 설정
    for word in encoded_words:
        context = [0] * block_size # [0,0,0]
        for ix in word + [0]:
            self.X.append(context.copy())
            self.Y.append(ix)
            context = context[1:] + [ix]

```

ex)

emma → [5,13,13,1]

(1) for ix=5 in [5,13,13,1,0]

context = [0,0,0]

X = [0,0,0]

Y = [5]

context = [0,0,5]

(2) for ix=13 in word + [0]

X = [0,0,5]

Y = [13]

X 입력	Y 정답	의미
[0, 0, 0]	5	... → e
[0, 0, 5]	13	..e → m
[0, 5, 13]	13	.em → m
[5, 13, 13]	1	emm → a
[13, 13, 1]	0	mma → .

3. MLP model 만들기

```
class MLPCharacterModel(nn.Module):
    def __init__(self, vocab_size, block_size, emb_dim=10,
                  hidden_dim=200):
        super().__init__()
        self.net = nn.Sequential(
            nn.Embedding(vocab_size, emb_dim),
            nn.Flatten(),
            nn.Linear(block_size * emb_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, vocab_size),
        )

    def forward(self, x):
        return self.net(x)

model = MLPCharacterModel(vocab_size, block_size)
logits = model(xb)
print("logits.shape:", logits.shape)
print("initial loss:", F.cross_entropy(logits, yb).item())
```

전체 shape 흐름

```
xb = [64, 3] = [batch, block]
```

```
Embedding = [64, 3, 10] (emb_dim = 10)
```

```
Flatten = [64, 30] -> [3,10]을 하나의 문자로 이어붙임.
```

```
Linear(30 → 200) : [64, 200] -> 은닉층은 200개
```

```
Tanh : [64, 200] -> Tanh은 비선형으로 변환하는 것
```

```
Linear(200 → 27) : [64, 27] -> Logits 만들기 (27은 vocab_size)
```

4. Train / Eval

```
def train_one_epoch(model, loader, optimizer, device):
    model.train()
    total_loss = 0.0
    for xb, yb in loader:
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)
        loss = F.cross_entropy(logits, yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * xb.size(0) # xb=[64,3]
    / xb.size(0)=64
    return total_loss / len(loader.dataset)
```

이름	의미
<code>model</code>	학습할 MLP 모델

이름	의미
loader	batch 단위로 데이터를 꺼내주는 DataLoader
optimizer	모델을 업데이트하는 도구
device	CPU 또는 GPU

전체 흐름

1. 입력 xb, 정답 yb 가져오기
2. 모델에 xb 넣어서 logits 얻기
3. logits와 yb 비교해서 loss 계산
4. 이전 gradient 지우기
5. loss.backward()로 gradient 계산
6. optimizer.step()으로 모델 업데이트
7. loss 누적

5. Train

```

device = "cuda" if torch.cuda.is_available() else "cpu" ##
하드웨어 = GPU 사
model = MLPCharacterModel(vocab_size, block_size).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-2)
# loss를 줄이는 방향으로 weight를 조정

for epoch in range(30):
    train_loss = train_one_epoch(model, loader, optimizer,
    device)
    if epoch % 5 == 0 or epoch == 29:
        print(f"epoch {epoch:2d} | train loss {train_loss:.
4f}")

```

6. Sampling

```

@torch.no_grad()
def sample(model, block_size, itos, device, num_samples=10,
max_len=20):
    model.eval()
    results = []
    for _ in range(num_samples):
        context = torch.zeros((1, block_size), dtype=torch.
long, device=device)
        out = []
        for _ in range(max_len):
            logits = model(context)
            probs = F.softmax(logits, dim=-1)
            ix = torch.multinomial(probs, num_samples=1) #
확률 기반으로 문자 하나 뽑기
            next_token = ix.item() # 뽑은 문자 next_token에
저장
            if next_token == 0: # 다음 token이 . 이면 더 이
상 할 필요 X
                break
            out.append(itos[next_token]) # 출력된 숫자 토큰을
문자로 변환해서 append (누적)
            context = torch.cat([context[:, 1:], ix], dim=
1) # context 업데이트
            results.append("".join(out))
        return results

sample(model, block_size, itos, device, num_samples=10)

```

예시 흐름

처음:

```
context: ...
```

모델이 **e** 를 뽑음.

```
out: e
context: ..e
```

다음:

```
context: ..e
```

모델이 **m**을 뽑음.

```
out: em
context: .em
```

다음:

```
context: .em
```

모델이 **m**을 뽑음.

```
out: emm
context: emm
```

다음:

```
context: emm
```

모델이 **a**를 뽑음.

```
out: emma
context: mma
```

다음:

```
context: mma
```

모델이 **.**을 뽑음.

생성 종료

최종 결과:

"emma"

Notebook 3

1. text data 전처리 (list 형태로 변환 → sort (정렬) → 숫자로 변환)

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from pathlib import Path

if not Path("input.txt").exists():
    !wget -q https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

text = open("input.txt", "r", encoding="utf-8").read()
chars = sorted(list(set(text)))
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}
vocab_size = len(chars)
data = torch.tensor([stoi[ch] for ch in text], dtype=torch.long)

print("text length:", len(text))
print("vocab_size:", vocab_size)
```



```
text: "First Citizen..."
data: [18, 47, 56, 57, 58, ...]
```

2. Sliding Window

구분	이름 MLP	Shakespeare MLP
데이터	짧은 이름 여러 개	긴 텍스트 하나
입력	이전 문자 3개	이전 문자 16개
정답	다음 문자	다음 문자
방식	이름 단위 context	sliding window
생성 결과	이름	문장/대사 비슷한 텍스트

```
class CharSequenceNextCharDataset(Dataset):
    def __init__(self, data, block_size):
        self.data = data
        self.block_size = block_size

    def __len__(self):
        return len(self.data) - self.block_size

    def __getitem__(self, idx):
        x = self.data[idx : idx + self.block_size]
        y = self.data[idx + self.block_size]
        return x, y

block_size = 16
dataset = CharSequenceNextCharDataset(data, block_size)
loader = DataLoader(dataset, batch_size=128, shuffle=True)

xb, yb = next(iter(loader))
print("xb.shape:", xb.shape)
print("yb.shape:", yb.shape)
print("decoded x:", ''.join(itos[i.item()] for i in xb[0]))
print("decoded y:", itos[yb[0].item()])
```

3. MLP model → model.shape([batch_size, vocab_size]) = logits

```
class MLPCharacterModel(nn.Module):
    def __init__(self, vocab_size, block_size, emb_dim=64,
                 hidden_dim=256):
        super().__init__()
        self.net = nn.Sequential(
            nn.Embedding(vocab_size, emb_dim),
            nn.Flatten(),
            nn.Linear(block_size * emb_dim, hidden_dim),
            nn.Tanh(),
            nn.Linear(hidden_dim, vocab_size),
        )

    def forward(self, x):
        return self.net(x)

model = MLPCharacterModel(vocab_size, block_size)
logits = model(xb)
print("logits.shape:", logits.shape)
print("initial loss:", F.cross_entropy(logits, yb).item())
```

입력 shape은:

```
xb.shape = [128, 16]
```

Embedding을 통과하면:

```
[128, 16] → [128, 16, 64]
```

Flatten 하면:

```
[128, 16, 64] → [128, 1024]
```

그다음 Linear를 통과:

```
1024 → 256 → vocab_size
```

출력은:

```
[128, vocab_size]
```

MLP vs GPT : self - attention

MLP:

```
토큰 → embedding → flatten → linear → 다음 토큰
```

GPT:

```
토큰 → token embedding + position embedding  
→ self-attention  
→ feed-forward  
→ 다음 토큰
```

Self-attention이 왜 필요한가?

MLP는 입력 16개를 그냥 한 번에 납작하게 펴.

```
[문자1, 문자2, 문자3, ..., 문자16]  
→ flatten
```

반면 self-attention은 각 위치의 문자가 다른 위치의 문자를 참고할 수 있게 해.

예를 들어:

```
ROMEO: I love Juliet
```

에서 **Juliet** 을 생성할 때, 모델은 앞의 **ROMEO** 나 **love** 같은 단어를 중요하게 볼 수 있어야 해.

Self-attention은 이런 질문을 하는 구조라고 보면 돼.

지금 위치의 문자를 예측할 때,
앞의 어떤 문자/토큰을 많이 참고해야 하지?

3. Train

```
def train_one_epoch(model, loader, optimizer, device, max_steps=None):
    model.train()
    total_loss, total_count = 0.0, 0
    for step, (xb, yb) in enumerate(loader):
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)
        loss = F.cross_entropy(logits, yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * xb.size(0)
        total_count += xb.size(0)
        if max_steps is not None and step + 1 >= max_steps:
            break
    return total_loss / total_count

device = "cuda" if torch.cuda.is_available() else "cpu"
model = MLPCharacterModel(vocab_size, block_size).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

for epoch in range(10):
    train_loss = train_one_epoch(model, loader, optimizer, device, max_steps=300)
    print(f"epoch {epoch:2d} | train loss {train_loss:.4f}")
```

4. Sampling

```
@torch.no_grad()
def sample_mlp(model, block_size, stoi, itos, device, start_text="ROME0:", max_new_tokens=300):
    model.eval()
    context = [0] * block_size
    for ch in start_text:
        if ch in stoi:
            context = context[1:] + [stoi[ch]] # context = [0,0,...,0,R,0,M,E,0,:]
    out = list(start_text)
    for _ in range(max_new_tokens):
        x = torch.tensor([context], dtype=torch.long, device=device)
        logits = model(x)
        probs = F.softmax(logits, dim=-1)
        ix = torch.multinomial(probs, num_samples=1).item()
        out.append(itos[ix])
        context = context[1:] + [ix]
    return "".join(out)

print(sample_mlp(model, block_size, stoi, itos, device, start_text="ROME0:", max_new_tokens=400))
```

Notebook 4

GPT - style dataset 구성 + sequence model 만들기

전체 흐름

1. Shakespeare 텍스트 불러오기
2. 문자 → 숫자 변환
3. GPT-style Dataset 만들기

4. position embedding이 있는 최소 sequence model 만들기
5. sequence loss 계산
6. 학습
7. 샘플링

1. 기초 설정

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from pathlib import Path

if not Path("input.txt").exists():
    !wget -q https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

text = open("input.txt", "r", encoding="utf-8").read() # text 파일 불러오기
chars = sorted(list(set(text))) # text 파일 리스트로 만들고 정렬하기
stoi = {ch: i for i, ch in enumerate(chars)} # text 파일에 있는 문자들 나열한 거 숫자로
itos = {i: ch for ch, i in stoi.items()} # 숫자를 문자로
vocab_size = len(chars)
data = torch.tensor([stoi[ch] for ch in text], dtype=torch.long) # text 데이터 숫자로 변환 후 tensor로 변환
```

2. GPT - style dataset

```
class NextTokenDataset(Dataset):
    def __init__(self, data, block_size):
        self.data = data
        self.block_size = block_size
```

```

def __len__(self):
    return len(self.data) - self.block_size # text 에서
추출한 학습할 X 데이터 개수

def __getitem__(self, idx):
    x = self.data[idx : idx + self.block_size]
    y = self.data[idx + 1 : idx + self.block_size + 1]
    return x, y

block_size = 32
dataset = NextTokenDataset(data, block_size) # text data ->
(X,Y) 데이터로 변환
loader = DataLoader(dataset, batch_size=64, shuffle=True) #
dataset을 batch size에 맞춰 불러오기

xb, yb = next(iter(loader))
print("xb.shape:", xb.shape)
print("yb.shape:", yb.shape)

```

3. Attention이 없는 최소 Sequence Model

```

class TinySequenceLM(nn.Module):

    def __init__(self, vocab_size, block_size, emb_dim=64):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, emb_dim)

        self.position_embedding = nn.Embedding(block_size, emb_dim)

        self.lm_head = nn.Linear(emb_dim, vocab_size)

    def forward(self, x):
        B, T = x.shape
        pos = torch.arange(T, device=x.device)
        tok = self.token_embedding(x) # (B, T,
C)

```

```

        pos = self.position_embedding(pos)[None] # (1, T,
C)
        h = tok + pos
        logits = self.lm_head(h) # (B, T,
V)
        return logits

model = TinySequenceLM(vocab_size, block_size)
logits = model(xb)
print("logits.shape:", logits.shape)

def sequence_cross_entropy(logits, targets):
    return F.cross_entropy(logits.transpose(1, 2), targets)
# entropy를 계산하기 위해선 [B,T,V] -> [B,T,V] 여야 함.

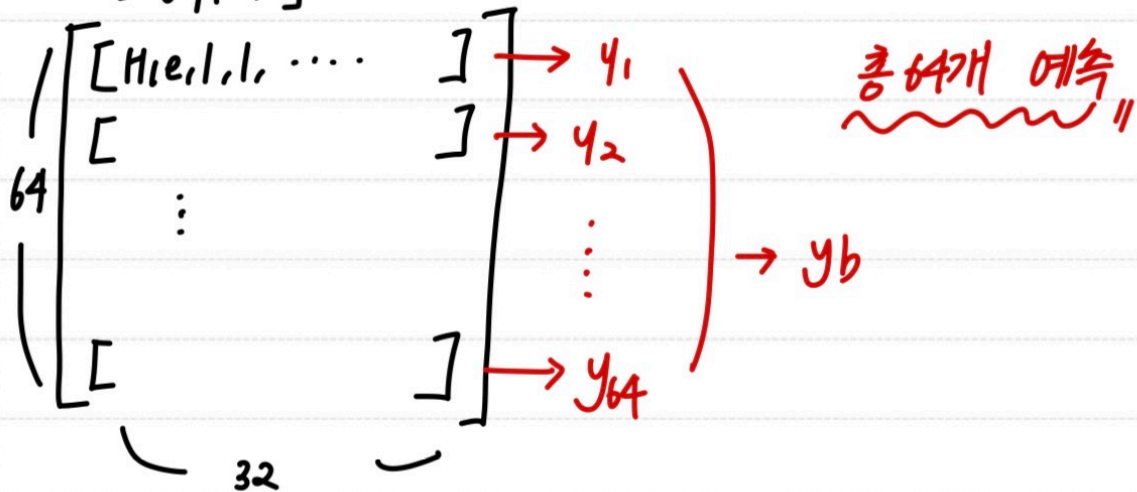
print("initial loss:", sequence_cross_entropy(logits, yb).i
tem())

```

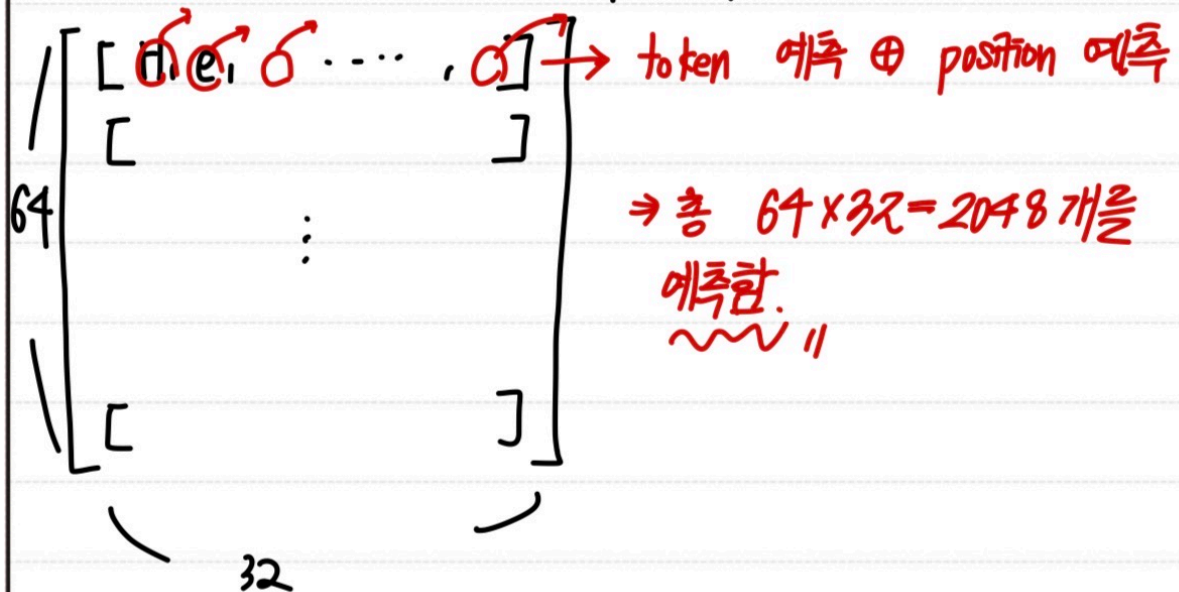

Sequence Model

• 기존 MLP Model

$x_b = [64, 32]$



• Sequence Model → Flatten 과정 제외



4. Train

```
def train_one_epoch(model, loader, optimizer, device, max_steps=None):
```

```

model.train()
total_loss, total_count = 0.0, 0
for step, (xb, yb) in enumerate(loader):
    xb, yb = xb.to(device), yb.to(device)
    logits = model(xb)
    loss = sequence_cross_entropy(logits, yb)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    total_loss += loss.item() * xb.size(0)
    total_count += xb.size(0)
    if max_steps is not None and step + 1 >= max_steps:
        break
return total_loss / total_count

device = "cuda" if torch.cuda.is_available() else "cpu"
model = TinySequenceLM(vocab_size, block_size).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

for epoch in range(100):
    train_loss = train_one_epoch(model, loader, optimizer,
    device, max_steps=300)
    print(f"epoch {epoch:2d} | train loss {train_loss:.4
    f}")

```

5. Samling

```

@torch.no_grad()
def sample_sequence_model(model, block_size, stoi, itos, de
vice, start_text="ROMEO:", max_new_tokens=300):
    model.eval()
    context = torch.zeros((1, block_size), dtype=torch.lon
g, device=device)
    for ch in start_text:
        if ch in stoi:
            ix = torch.tensor([[stoi[ch]]], device=device)
            context = torch.cat([context[:, 1:], ix], dim=

```

```

1)
out = list(start_text)
for _ in range(max_new_tokens):
    logits = model(context)
    logits = logits[:, -1, :]
    probs = F.softmax(logits, dim=-1)
    ix = torch.multinomial(probs, num_samples=1)
    out.append(itos[ix.item()])
    context = torch.cat([context[:, 1:], ix], dim=1)
return "".join(out)

print(sample_sequence_model(model, block_size, stoi, itos,
device, start_text="ROME0:", max_new_tokens=400))

```

MLP vs Sequence vs Attention

일반 MLP : 입력 전체를 한 덩어리로 보고 (Flatten) → 다음 글자 예측

Sequence model : 각 위치마다 → 다음 글자 예측 ex. [h,e,l,l,o] → [e,l,l,o,공백]

Self-attention : 각 위치가 이전 위치들 중 어디를 참고할지 스스로 정함 ex.

Embedding의 원리

Key, Value, Query

Key, Value, Query

Notebook 5

Masked self - attention

기존 attention은 자기 자신의 위치만 알 수 있음 → self-attention은 자기 자신의 위치 + 이전에 위치한 문자들도 알 수 있음

```
Query = Q = [B,T,C]
Key = K = [B,T,C]
Value = V = [B,T,C]
```

```
Q @ K^T = [B,T,T] -> [T,T] = 관계 행렬
softmax (Q@K^T / sqrt(C)) @ V = [B,T,C]
```

1. 기초 설정

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from pathlib import Path

if not Path("input.txt").exists():
    !wget -q https://raw.githubusercontent.com/karpathy/char-rnn/master/data/tinyshakespeare/input.txt

text = open("input.txt", "r", encoding="utf-8").read()
chars = sorted(list(set(text)))
stoi = {ch: i for i, ch in enumerate(chars)}
itos = {i: ch for ch, i in stoi.items()}
vocab_size = len(chars)
data = torch.tensor([stoi[ch] for ch in text], dtype=torch.long)

class NextTokenDataset(Dataset):
    def __init__(self, data, block_size):
        self.data = data
```

```

        self.block_size = block_size
    def __len__(self):
        return len(self.data) - self.block_size
    def __getitem__(self, idx):
        x = self.data[idx : idx + self.block_size]
        y = self.data[idx + 1 : idx + self.block_size + 1]
        return x, y

block_size = 32
dataset = NextTokenDataset(data, block_size)
loader = DataLoader(dataset, batch_size=64, shuffle=True)
xb, yb = next(iter(loader))

```

2. Single-head masked self-attention → forward 이후 부분은 위 K,Q,V 참고

```

class SingleHeadSelfAttention(nn.Module):
    def __init__(self, emb_dim, block_size):
        super().__init__()
        self.key = nn.Linear(emb_dim, emb_dim, bias=False)
        self.query = nn.Linear(emb_dim, emb_dim, bias=False)
        self.value = nn.Linear(emb_dim, emb_dim, bias=False)

        self.register_buffer("tril", torch.tril(torch.ones(
            block_size, block_size))) # 각 token이 다음 token 보지 못하게
            만들기

    def forward(self, x):
        B, T, C = x.shape
        k = self.key(x)
        q = self.query(x)
        v = self.value(x)

        wei = q @ k.transpose(-2, -1) * (C ** -0.5)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float(
            "-inf"))

```

```
wei = F.softmax(wei, dim=-1)

out = wei @ v
return out
```

causal mask 만들기

```
self.register_buffer("tril", torch.tril(torch.ones(block_size, block_size)))
```

→ 미래 token을 못 보게 막는 표를 만듦

`block_size = 5` 라고 하면:

```
torch.ones(5,5)
```

는:

```
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
1 1 1 1 1
```

그런데 `torch.tril` 을 적용하면 아래쪽 삼각형만 남음.

```
1 0 0 0 0
1 1 0 0 0
1 1 1 0 0
1 1 1 1 0
1 1 1 1 1
```

이 표의 의미는:

```
0번 위치는 0번까지만 볼 수 있음
1번 위치는 0,1번 볼 수 있음
2번 위치는 0,1,2번 볼 수 있음
3번 위치는 0,1,2,3번 볼 수 있음
```

3. Attention포함 최소 모델

```
class TinyAttentionLM(nn.Module):
    def __init__(self, vocab_size, block_size, emb_dim=64):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, emb_dim)
        self.position_embedding = nn.Embedding(block_size, emb_dim)
        self.attn = SingleHeadSelfAttention(emb_dim, block_size)
        self.lm_head = nn.Linear(emb_dim, vocab_size)

    def forward(self, x):
        B, T = x.shape
        pos = torch.arange(T, device=x.device)
        tok = self.token_embedding(x)
        pos = self.position_embedding(pos)[None]
        h = tok + pos
        h = self.attn(h)
        logits = self.lm_head(h)
        return logits

model = TinyAttentionLM(vocab_size, block_size)
logits = model(xb)
print("logits.shape:", logits.shape)
```

```
xb
[64, 32]
```

↓ token embedding

```
tok
[64, 32, 64]
```

↓ position 더하기

```

h
[64, 32, 64]

↓ self-attention

h
[64, 32, 64]

↓ lm_head

logits
[64, 32, vocab_size]

```

4. Train

```

def sequence_cross_entropy(logits, targets):
    return F.cross_entropy(logits.transpose(1, 2), targets)
#[B,T,V] -> [B,V,T]

def train_one_epoch(model, loader, optimizer, device, max_steps=None):
    model.train()
    total_loss, total_count = 0.0, 0
    for step, (xb, yb) in enumerate(loader):
        xb, yb = xb.to(device), yb.to(device)
        logits = model(xb)
        loss = sequence_cross_entropy(logits, yb)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * xb.size(0)
        total_count += xb.size(0)
        if max_steps is not None and step + 1 >= max_steps:
            break
    return total_loss / total_count

```



```

device = "cuda" if torch.cuda.is_available() else "cpu"
model = TinyAttentionLM(vocab_size, block_size).to(device)
optimizer = torch.optim.AdamW(model.parameters(), lr=1e-3)

for epoch in range(100):
    train_loss = train_one_epoch(model, loader, optimizer,
    device, max_steps=300)
    print(f"epoch {epoch:2d} | train loss {train_loss:.4
    f}")

```

Gradient는 무엇인가?

loss를 줄이려면 각 가중치를 어느 방향으로 얼마나 바꿔야 하는지 알려주는 값

5. Sampling

```

@torch.no_grad()
def sample_attention_model(model, block_size, stoi, itos, device,
    start_text="ROME0:", max_new_tokens=300):
    model.eval()
    context = torch.zeros((1, block_size), dtype=torch.long, device=device)
    for ch in start_text:
        if ch in stoi:
            ix = torch.tensor([[stoi[ch]]], device=device)
            context = torch.cat([context[:, 1:], ix], dim=1)

    out = list(start_text)
    for _ in range(max_new_tokens):
        logits = model(context) # logits = [B,T,V]
        logits = logits[:, -1, :] # 예측은 T의 가장 마지막 문자 다음에 올 친구로 함
        probs = F.softmax(logits, dim=-1) # 마지막에 오는 친구 다음에 올 친구들을 확률화
        ix = torch.multinomial(probs, num_samples=1) # 확률

```

```

을 보고 하나 고름
        out.append(itos[ix.item()]) # 고른 숫자 값을 문자로 바
꿈
        context = torch.cat([context[:, 1:], ix], dim=1) #
context 업데이트
        return "".join(out)

print(sample_attention_model(model, block_size, stoi, itos,
device, start_text="ROME0:", max_new_tokens=400))

```

Notebook 6

embedding → (layer norm → multi head self attention → layer norm →
FeedForward → lm_head)

1. Multi-head attention

```

class Head(nn.Module):
    def __init__(self, emb_dim, head_size, block_size, drop
out=0.1):
        super().__init__()
        self.key = nn.Linear(emb_dim, head_size, bias=False)
        self.query = nn.Linear(emb_dim, head_size, bias=False)
        self.value = nn.Linear(emb_dim, head_size, bias=False)
        self.register_buffer("tril", torch.tril(torch.ones
(block_size, block_size)))
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        B, T, C = x.shape
        k = self.key(x)
        q = self.query(x)

```

```

        v = self.value(x)
        wei = q @ k.transpose(-2, -1) * (k.size(-1) ** -0.5)
5)
        wei = wei.masked_fill(self.tril[:T, :T] == 0, float("-inf"))
        wei = F.softmax(wei, dim=-1)
        wei = self.dropout(wei)
        out = wei @ v
        return out

class MultiHeadAttention(nn.Module):
    def __init__(self, emb_dim, num_heads, block_size, dropout=0.1):
        super().__init__()
        head_size = emb_dim // num_heads
        self.heads = nn.ModuleList([Head(emb_dim, head_size, block_size, dropout) for _ in range(num_heads)])
        self.proj = nn.Linear(emb_dim, emb_dim)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x):
        out = torch.cat([h(x) for h in self.heads], dim=-1)
        out = self.proj(out)
        out = self.dropout(out)
        return out

```

2. FeedForward + Block

```

class FeedForward(nn.Module):
    def __init__(self, emb_dim, dropout=0.1):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(emb_dim, 4 * emb_dim),
            nn.ReLU(),
            nn.Linear(4 * emb_dim, emb_dim),
            nn.Dropout(dropout),
        )

```

```

def forward(self, x):
    return self.net(x)

class Block(nn.Module):
    def __init__(self, emb_dim, num_heads, block_size, dropout=0.1):
        super().__init__()
        self.ln1 = nn.LayerNorm(emb_dim)
        self.sa = MultiHeadAttention(emb_dim, num_heads, block_size, dropout)
        self.ln2 = nn.LayerNorm(emb_dim)
        self.ffwd = FeedForward(emb_dim, dropout)

    def forward(self, x):
        x = x + self.sa(self.ln1(x)) # residual connection
        x = x + self.ffwd(self.ln2(x))
        return x

```

- FeedForward → 표현을 정교화하기 위해 embedding dimension을 4배로 늘린 것
- Block → 더 깊게 학습할 수 있게 하기 위해 Block 을 늘린 것 (self-attention으로 여러번 학습)

3. Tiny GPT

```

class TinyGPT(nn.Module):
    def __init__(self, vocab_size, block_size, emb_dim=128, num_heads=4, num_layers=4, dropout=0.1):
        super().__init__()
        self.token_embedding = nn.Embedding(vocab_size, emb_dim)
        self.position_embedding = nn.Embedding(block_size, emb_dim)
        self.blocks = nn.Sequential(*[
            Block(emb_dim, num_heads, block_size, dropout)
            for _ in range(num_layers)
        ])
        self.ln_f = nn.LayerNorm(emb_dim)

```

```

        self.lm_head = nn.Linear(emb_dim, vocab_size)

    def forward(self, x):
        B, T = x.shape
        pos = torch.arange(T, device=x.device)
        tok = self.token_embedding(x)
        pos = self.position_embedding(pos)[None]
        h = tok + pos
        h = self.blocks(h) # Embedding 후 여러 블록 통과시키면
서 self - attention
        h = self.ln_f(h) # LayerNormalize (층 정규화, 안정화
시키기)
        logits = self.lm_head(h)
        return logits

model = TinyGPT(vocab_size, block_size)
logits = model(xb)
print("logits.shape:", logits.shape)

```